About UniConnected **1**
Platform & Tech Stack **2**
Development & Staging Environment **3**
Project Scope: Employment Opportunity **3**
Student Experience 3
Organisation Experience 4
What You Will Build 4
Additional Tasks 5
Key Deliverables & Timeline **5**
Deliverables 5
Collaboration & Workflow **6**
How We Work 6
Expectations 7
Resources & Next steps **7**
Resources 7
Next Steps 8

About UniConnected

UniConnected is reimagining how university communities connect. Our platform brings together students, alumni, academics, and organisations to explore and discover opportunities that enrich careers and lives.

We are building a space where connections happen seamlessly and lead to meaningful outcomes. While the possibilities are endless, we are starting with key opportunity types:

- **Employment**: Organisations (companies, SMEs, startups, schools, universities, etc.) can connect with students and alumni for casual, part-time, or full-time roles.

- **Curriculum-linked opportunities**: Students can find internships or placements tied to their course requirements.

- **Mentoring & Coaching**: Students can connect with alumni, professionals, academics, or peers for guidance and support.

- **Research, Consulting & Entrepreneurship**: Pathways for collaboration and innovation within and beyond the university.

The platform is already live with a pilot: the curriculum-linked opportunity **Masters of Teaching Secondary Internship (MTSI)** at the University of Melbourne. The next major feature is the **Employment Opportunity module** — and you will help us design and build it.

Platform & Tech Stack

UniConnected is built as a modern **web platform** with a **Next.js frontend** and a **Django REST Framework backend**. The system is modular and scalable, allowing us to add new opportunity types (like Employment) without disrupting existing ones.

- **Frontend**:
  - Built with **Next.js (React framework) + TypeScript**
  - **Chakra UI** for styling and components
  - Communicates with the backend through REST APIs
  - Deployed on **AWS Amplify**

- **Backend**:
  - **Django + Django REST Framework (DRF)**
  - **PostgreSQL** as the primary database
  - Deployed on **AWS ECS**
  - Provides APIs for authentication, onboarding, user management, and opportunities

- **Other Tools**:
  - **Swagger / drf-spectacular** for API documentation
  - **GitHub** for version control and collaboration
  - **Docker** for consistent local development environments
  - **Google APIs** for address autocomplete and location data
  - **CI/CD pipelines** for automated builds and deployments

Development & Staging Environment

The UniConnected platform is already live with a pilot: the **Masters of Teaching Secondary Internship (MTSI)** at the University of Melbourne. For your project, you will primarily work in our **staging environment**, which mirrors production but is safe for testing and development.

Staging URL: https://staging.uniconnected.com/

Username: staging
Password: $8S7*a8AHE#4

You are encouraged to **sign up and test** both as a **Student** and an **Industry Partner (Organisation)** to understand the user experience.

Important: MTSI is an *invite-only* opportunity. This means that when you first log in, the Discover page will appear empty. To explore MTSI, you need to invite yourself as both a student and an organisation using the **Coordinator account**:

Username: coordinator@uniconnected.demo
Password: StrongPassword123

Once logged in as Coordinator:

1. Go to **Manage Students > Add Students** → invite your student account.
2. Go to **Manage Organisations > Add Organisation** → invite your organisation account.

This will give you full access to the MTSI opportunity flow.

Note: The staging environment is in **active development**. Bugs and unfinished features are expected. Please report any issues you encounter so we can track and fix them.

Project Scope: Employment Opportunity

Your project is to design and implement the **Employment Opportunity module** in UniConnected. This will extend the platform beyond curriculum-linked pilots like MTSI to allow students and organisations to connect for real employment.

Student Experience

- A **Student** signs up and completes onboarding. If they have a **University of Melbourne email account** (e.g. @student.unimelb.edu.au), they can join the Employment Opportunity as an employee. *(Support for other universities will be added later.)*

- Students will answer a short **Employment Questionnaire**:

    1. **Type of employment**: Casual/Part-time, Full-time, Graduate Role
    2. **Duration**: Short-term, Permanent
    3. **Sector**: Engineering, Education, etc. *(same list as Organisation onboarding)*
    4. **Open to work as**: e.g. Music Tutor, IT, Marketing

- Once joined, students see a **Discover Employment page** (accessible via a new dropdown menu if they are in multiple opportunities). Here they can **filter/search organisations** and reach out to those that allow contact.
- For clarity - this is NOT like SEEK or LinkedIN- there is no job advertising on the platform. The purpose is to focus on Opportunities and Connections

Organisation Experience

- An **Organisation** signs up and can join the Employment Opportunity as an employer.

- No questionnaire is required.

- Organisations also see a **Discover Employment page**, where they can **filter/search students** and contact them.

- Access is limited by a **paywall**: filters and results (e.g. by university, employment type) may be restricted depending on subscription level.

What You Will Build

As part of this project, you will:

1. **Create the Employment Opportunity** with the appropriate models, fields, and questionnaire.

2. Allow **open enrollment** (unlike MTSI, which is invite-only). Students and organisations should be able to join after onboarding. Students will be restricted based on the URL of their email and whether their university is a UniConnected partner.

3. Introduce a **University model** so that only students with UniMelb email domains can initially join.

4. **Generalise the Discover page** into a reusable component, with a navigation dropdown for multiple opportunities.

5. Build the **Discover Employment page** with filters and search for both students and organisations.

6. Implement a **paywall system** for organisations, limiting filters/results based on subscription level.

7. Provide **APIs, models, and Swagger documentation**, along with **unit tests** to ensure reliability.

Additional Tasks

In addition to the Employment Opportunity, you may be asked to support our team with improvements to core functionality, including:

- Enhancing **matching criteria** in Discover (e.g. location proximity).

- Improving the **Organisation model**.

- Fixing bugs and general platform improvements.

Key Deliverables & Timeline

Your primary deliverable is a fully functional **Employment Opportunity module**, ready for release by the end of your semester. To achieve this, we will follow an **agile sprint structure**, with four 2-week sprints (8 weeks total).

Deliverables

By the end of the project, you should deliver:

1. **Employment Opportunity created** with correct models, fields, and questionnaires.
2. **Open enrollment** for students (restricted to UniMelb emails initially) and organisations.
3. **Discover Employment page** for both students and organisations, including filters, search, and contact functionality.
4. **Reusable Discover component** with dropdown navigation for multiple opportunities.
5. **Paywall implementation** for organisations (limiting filters/results by subscription level).
6. **Supporting APIs, Swagger documentation, and unit tests** to ensure maintainability.
7. **Bug fixes and platform improvements** (as needed to support MTSI and core functionality).

Collaboration & Workflow

You will be working in **two teams**, but the goal is to function as **one project group** building the Employment Opportunity module. Collaboration and alignment across teams is essential.

How We Work

1. **Sprint Planning**
    - At the start of each sprint, we will design **user stories** together.
    - Each user story is then broken down into **clear technical tasks** (backend, frontend etc.).
2. **Task Ownership**
    - Tasks will be assigned across both teams.
    - The key is to make sure both teams stay aligned and no one works in isolation.
3. **Development & Pull Requests**
    - Each task should be developed on a **feature branch**.

    - When a task is complete, a **pull request (PR)** is created into the develop branch.
    - PRs should include a **clear description** of what was implemented.
4. **Code Reviews**
    - Each PR must be reviewed by at least **one member from the other team**.
    - This ensures both teams understand the full system and align on implementation decisions.
    - I will provide the **final review and merge** into the develop branch.
5. **Quality & Testing**
    - Each feature should include **unit tests** and be tested in the **staging environment**.
    - Bugs or issues discovered should be logged and fixed promptly.
6. **Communication**
    - Use GitHub issues and pull requests for technical discussions.
    - Use Slack for quick questions and coordination.
    - Keep sprint progress visible through a shared backlog/board (JIRA)

Expectations

- Stay aligned as **one team with two squads**.
- Write clear, maintainable code with documentation.
- Support each other with reviews and testing.
- By the end of each sprint, we should have **working, tested features**, ready to demo.

Resources & Next steps

Resources

- **API Documentation**: https://dev.uniconnected.com/api/docs/

- **GitHub Repositories**:

    - Backend: uniconnected-django
    - Frontend: uniconnected2-frontend

- **Key Contact**: I (Bassam) am your primary point of contact. Please communicate with me directly at **bassam@uniconnected.com**.

Next Steps

1. **GitHub Access**

    - Please send me the GitHub accounts of all team members so I can grant you repository access.

2. **Slack & JIRA Access**

    - Each team should nominate **one representative** to join our Slack workspace and JIRA board.

    - This will:

        - Keep communication streamlined and avoid noise.

        - Keep our JIRA account bill manageable.

    - I will assign tasks in JIRA to the team representative, who can then redistribute tasks internally.

    - Please provide me with the **email address** of the nominated representative from each team so I can add them to Slack and JIRA.

3. **Skills Matrix**

    - As discussed, please prepare a **skills matrix** for each team. This should outline your familiarity with our tech stack (Next.js/TypeScript, Chakra UI, Django/DRF, PostgreSQL, Docker, AWS, GitHub).

    - Include any additional strengths (e.g. UI/UX design, testing, DevOps, data analysis) so we can make best use of your skills.